PhD Research and First Paper Project Plan

ZX Calculus + Stim + Graph-Theoretic Quantum Compilation

Prepared as a 6-month roadmap for a first publishable PhD project and long-term thesis direction

1. Executive Summary

Your PhD direction should combine applied mathematics, quantum computing, and serious software development. The recommended thesis theme is graph-theoretic quantum compilation using ZX-calculus, with applications to stabilizer circuits and quantum error correction using Stim.

The first 6-month project should be focused enough to produce a paper and a working software prototype. The recommended first paper is:

Graph-Feature-Guided Optimization of Stim Stabilizer Circuits using ZX-Inspired Rewriting

The goal is not to invent the whole ZX-calculus. The goal is to build a practical software pipeline that takes stabilizer/QEC circuits in Stim, converts their structure into graph objects, computes graph features, applies ZX-inspired transformations or rewrite-selection heuristics, and benchmarks whether the circuit or detector model becomes cheaper.

2. Long-Term PhD Research Program

A strong PhD thesis should not be a collection of unrelated papers. It should be a sequence of papers that build one coherent research identity. Your identity can be:

```
Graph theory + ZX calculus + stabilizer circuits + quantum error correction + compiler software
```

2.1 Possible PhD Thesis Title

Graph-Theoretic and ZX-Calculus Methods for Fault-Tolerant Quantum Compilation

2.2 Proposed Paper Sequence

Paper	Topic	Main contribution	Software output
1	Graph-feature-guided optimization of Stim stabilizer circuits	First publishable algorithm and benchmark study	Python package: stim-zx-optimizer
2	Symmetry-aware ZX rewriting for stabilizer/QEC circuits	Use automorphisms or orbit representatives to reduce rewrite search	Symmetry module using NetworkX/nauty-style tools
3	Cost-aware ZX compilation for fault-tolerant circuits	Optimize CNOT/CZ depth, measurement rounds, detector-model size	Compiler cost model and benchmark suite
4	ZX-calculus analysis of QEC syndrome extraction circuits	Represent syndrome extraction as ZX/graph transformations	Stim/QEC circuit analyzer
5	Graph invariants predicting ZX optimization difficulty	Mathematical/experimental study of treewidth, rank, degree, symmetry, graph size	Dataset and analysis notebooks
6	Open-source graph-based quantum compiler	Integrate parser, graph conversion, optimizer,	Professional GitHub project

	framework	extraction, benchmarking	

3. First 6-Month Project

3.1 Project Title

Graph-Feature-Guided Optimization of Stim Stabilizer Circuits using ZX-Inspired Rewriting

3.2 Main Research Question

Can graph-theoretic features and ZX-inspired rewrite ideas improve the optimization or analysis of stabilizer and QEC circuits written in Stim format?

3.3 More Precise Sub-Questions

1. How can a Stim circuit be represented as a graph suitable for ZX-inspired analysis?
2. Which graph features predict circuit cost: vertices, edges, degree distribution, connected components, bipartite structure, measurement structure, and detector-model size?
3. Can a cost function guide local transformations better than a fixed simplification strategy?
4. For stabilizer/QEC circuits, what cost should be minimized: CNOT/CZ count, depth, number of measurements, detector error model size, sampling time, or logical error rate?
5. Does the optimization preserve the detector error model or logical behavior of the circuit?

3.4 Why Stim Instead of Qiskit?

Stim is a high-performance simulator for stabilizer circuits, especially quantum error correction circuits. This aligns directly with your interest in stabilizers, Pauli measurements, syndrome extraction, and fault-tolerant circuits. PyZX is a Python tool for ZX-calculus circuit rewriting, visualization, and automated optimization. PyMatching is a fast Python/C++ decoder for quantum error correction using minimum-weight perfect matching. Together they form a strong research and software stack for your PhD.

Tool	Role in your project
Stim	Create and simulate stabilizer/QEC circuits; produce detector error models; sample syndromes.
Sinter	Run repeated Stim experiments and collect logical error-rate data.
PyMatching	Decode detector samples and evaluate QEC performance.
PyZX	Study ZX-calculus representations and rewrite strategies.
NetworkX	Represent graph structures and compute graph features.
pytest	Build professional tests for your software.
Matplotlib/Pandas	Analyze and plot benchmark results.

4. Complete Software Pipeline

The core pipeline should be simple at first and then become more advanced. Do not start with a giant compiler. Start with a small working pipeline.

```
Stim circuit (.stim)
    ↓
Parse circuit instructions
    ↓
Build graph representation
```

```
    ↓
Compute graph features
    ↓
Apply cost-aware or ZX-inspired transformations
    ↓
Return transformed Stim circuit
    ↓
Use Stim to check detector error model / sampling behavior
    ↓
Use PyMatching/Sinter for QEC benchmarks
    ↓
Compare before and after optimization
```

4.1 Minimum Working Version

- Input: small Clifford/stabilizer circuits in Stim syntax.
- Graph: qubits are nodes, two-qubit gates are edges, measurements are special output nodes or labels.
- Features: number of gates, number of two-qubit gates, depth estimate, connected components, degree statistics.
- Transformation: reorder commuting gates where safe, detect redundant Clifford patterns, or choose simplification candidates.
- Verification: compare detector error models or sampled statistics before and after transformation.

4.2 Advanced Version

- Represent a closer ZX-style graph with spider types, phases, boundaries, and measurement nodes.
- Use PyZX for circuits that can be converted into supported formats, and compare with your own graph representation.
- Add graph automorphism or symmetry grouping to avoid testing equivalent rewrite positions.
- Benchmark on surface-code memory circuits generated by Stim.
- Measure detector error model size and decoding performance using PyMatching/Sinter.

5. What You Need to Study

Area	What to learn	Why it matters
Stabilizer theory	Pauli strings, Clifford gates, stabilizer groups, tableau simulation	Stim and QEC are built around stabilizer circuits.
ZX-calculus	Z/X spiders, spider fusion, bialgebra, Hopf, graph-like form, local complementation, pivoting	This is the mathematical language of your compiler.
Graph states	Stabilizer description of graph states, local Clifford equivalence, local complementation	Connects stabilizers with graph transformations.
Quantum error correction	Syndrome extraction, detectors, logical error rate, surface code, MWPM decoding	Gives your project real fault-tolerance applications.
Compiler optimization	IR, rewrite rules, cost functions, benchmarks, equivalence checking	Turns mathematics into practical quantum software.
Software engineering	Git, tests, modules, documentation, profiling	Required for quantum software developer jobs.

6. Six-Month Step-by-Step Plan

Month	Main goal	Deliverable

1	Foundations and environment	VS Code project, GitHub repo, Stim/PyZX/PyMatching installed, small examples running.
2	Graph representation	Stim-to-graph parser and graph feature extractor.
3	First optimizer	Cost function and first transformation strategy.
4	Benchmarking	Benchmark suite with tables and plots.
5	Improvement and validation	Better transformations, tests, documentation, detector-model checks.
6	Paper writing	Complete manuscript, cleaned code, reproducible experiments.

6.1 Month 1: Foundations and Setup

Week	Tasks	Output
1	Install VS Code, Python, Git; create virtual environment; install stim, sinter, pymatching, pyzx, networkx, pytest. Run first Stim circuit.	Working environment and GitHub repository.
2	Study Stim circuits, measurements, detectors, observables. Write examples of Bell pair, GHZ state, repetition code.	notebooks/01_stim_basics.ipynb and src/stim_examples.py.
3	Study stabilizer theory: Pauli strings, Clifford conjugation, stabilizer groups, measurement.	Personal notes: stabilizer_summary.md.
4	Study ZX basics and PyZX examples: spiders, graph-like diagrams, simplification.	notebooks/02_pyzx_basics.ipynb.

6.2 Month 2: Build the Stim-to-Graph Layer

Week	Tasks	Output
5	Define your internal graph model. Decide node types: qubit, gate, measurement, detector. Decide edge types.	src/graph_model.py.
6	Parse simple Stim circuits. Extract gates H, S, CX, CZ, M, R, DETECTOR, OBSERVABLE_INCLUDE.	src/stim_parser.py.
7	Compute graph features: number of nodes/edges, degree, connected components, two-qubit interaction graph, measurement count.	src/graph_features.py.
8	Write tests for parser and feature extractor.	tests/test_parser.py and tests/test_features.py.

6.3 Month 3: First Optimization Algorithm

Week	Tasks	Output
9	Define cost functions: gate_count, two_qubit_count, depth_estimate, detector_model_size.	src/cost_functions.py.
10	Implement safe local transformations for simple Clifford patterns. Begin with conservative transformations	src/transformations.py.

	only.	
11	Implement greedy selection: evaluate candidate transformations and choose the best cost decrease.	src/optimizer.py.
12	Validate equivalence using Stim where possible: compare detector error model, sampled statistics, or known stabilizer behavior.	src/validate.py.

6.4 Month 4: Benchmarking

Week	Tasks	Output
13	Create benchmark circuits: Bell/GHZ, random Clifford circuits, repetition-code circuits, surface-code memory circuits.	benchmarks/circuits/.
14	Automate runs: baseline vs optimized. Save metrics in CSV.	src/benchmark.py and results/*.csv.
15	Plot results: gate count, CNOT/CZ count, depth, runtime, detector model size.	notebooks/03_results_plots.ipynb.
16	Analyze failures: where does the method help, where does it not help?	results_analysis.md.

6.5 Month 5: Improve and Harden the Software

Week	Tasks	Output
17	Add better transformations or rewrite-selection heuristics. Consider symmetry grouping if time permits.	optimizer_v2.py.
18	Improve validation: more tests, more random circuits, compare detector models carefully.	tests/test_validation.py.
19	Clean repository: README, installation guide, examples, command-line script.	Professional GitHub repository.
20	Freeze experiments and generate final tables.	Final results folder.

6.6 Month 6: Write the Paper

Week	Tasks	Output
21	Write introduction, motivation, and related work.	paper/introduction.tex.
22	Write method section: graph representation, features, cost function, optimizer.	paper/method.tex.
23	Write experiments and results.	paper/experiments.tex and figures.
24	Write discussion, limitations, conclusion, and future work. Submit to supervisor for feedback.	Complete paper draft.

7. Start Tomorrow: Day 1 Checklist

6. Install VS Code if you do not already have it.
7. Install Python 3.11 or newer and Git.
8. Create a folder called zx-stim-phd.

9. Open the folder in VS Code.
10. Create a virtual environment.
11. Install the required packages.
12. Create the project structure.
13. Run a first Stim circuit.
14. Create your first Git commit.

```
mkdir zx-stim-phd
cd zx-stim-phd
code .
python -m venv .venv
# Windows:
.venv\Scripts\activate
# Mac/Linux:
source .venv/bin/activate
pip install stim sinter pymatching pyzx networkx numpy pandas matplotlib pytest jupyter
pip freeze > requirements.txt
```

7.1 First Python File

```
# src/main.py
import stim

circuit = stim.Circuit("""
H 0
CX 0 1
M 0 1
""")

print(circuit)
print("Number of instructions:", len(circuit))
```

8. Recommended Repository Structure

```
zx-stim-phd/
├── README.md
├── requirements.txt
├── pyproject.toml
├── src/
│   ├── stim_zx_optimizer/
│   │   ├── __init__.py
│   │   ├── stim_parser.py
│   │   ├── graph_model.py
│   │   ├── graph_features.py
│   │   ├── cost_functions.py
│   │   ├── transformations.py
│   │   ├── optimizer.py
│   │   ├── validate.py
│   │   └── benchmark.py
├── tests/
├── notebooks/
├── benchmarks/
│   └── circuits/
├── results/
```

```
├── paper/
└── docs/
```

9. First Paper Outline

Section	Content
Abstract	One paragraph: problem, method, experiments, main result.
1. Introduction	Why QEC/stabilizer circuit optimization matters; why graph/ZX methods are promising.
2. Background	Stim, stabilizer circuits, ZX calculus, graph representations, QEC detector models.
3. Method	Stim-to-graph conversion, graph features, cost function, rewrite/optimization strategy.
4. Implementation	Software architecture and reproducibility details.
5. Experiments	Benchmark circuits, metrics, baseline methods.
6. Results	Tables and plots comparing baseline and optimized circuits.
7. Discussion	Where the method works, limitations, correctness/equivalence issues.
8. Future Work	Automorphisms, deeper ZX integration, surface-code/lattice-surgery compilation.
9. Conclusion	Main contribution and next steps.

10. Success Criteria for the First Paper

- A clean software package that other people can run.
- A clear graph representation of Stim stabilizer/QEC circuits.
- At least one cost-aware optimization strategy.
- A benchmark suite with reproducible results.
- Evidence that the method improves at least one metric on some circuit families, even if it does not always win.
- A careful explanation of limitations and future improvements.

11. Risks and How to Avoid Them

Risk	Solution
The project becomes too broad	Do not try to build a universal compiler in paper 1. Focus on stabilizer/QEC circuits.
Equivalence checking is difficult	Start with conservative transformations and use Stim detector models/sampling for validation.
ZX conversion is hard	Begin with graph-inspired and ZX-inspired representations; integrate PyZX gradually.
No improvement in benchmarks	Report where it works and where it fails; negative/partial results can still guide paper 2.
Software becomes messy	Use tests, modules, documentation, and Git from day 1.

12. Starting References

Use these as starting points. The exact bibliography should be expanded during Month 1.

- Stim GitHub: high-performance simulation and analysis of quantum stabilizer circuits, especially QEC circuits.
- Stim paper: Stim: a fast stabilizer circuit simulator, arXiv:2103.02202.

- PyZX GitHub and documentation: Python tool for ZX-calculus, visualization, and automated rewriting of quantum circuits.
- PyZX paper: PyZX: Large Scale Automated Diagrammatic Reasoning, arXiv:1904.04735.
- PyMatching GitHub and paper: fast Python/C++ MWPM decoder for quantum error-correcting codes.
- Picturing Quantum Processes by Coecke and Kissinger: main conceptual reference for ZX-calculus.
- Nielsen and Chuang: standard reference for quantum computing, stabilizers, and QEC.

13. Final Advice

Your first paper should be practical and focused. The deeper mathematics - automorphisms, canonical forms, graph invariants, and fault-tolerant compilation - can become papers 2, 3, and 4. For now, your mission is to build a working research pipeline and produce a clean benchmark paper.

A good first paper does not need to solve all ZX compilation. It needs to make one clear contribution, implemented well, explained carefully, and evaluated honestly.